Secure File Management System

Technical Architecture & Design Document

Full-Stack RBAC File Platform
Spring Boot 3.2 + React 18 + MySQL 8.0
Version 1.0 | February 2026

Table of Contents

1. Project Overview
2. Technology Stack
3. System Architecture
4. Entity Relationship Diagram
5. Authentication & JWT Flow
6. Role-Based Access Control
7. File CRUD Operations
8. File Upload & Access Flow
9. Frontend-Backend Communication
10. API Endpoints Reference
11. Database Schema
12. Security Architecture
13. Frontend Pages & Components
14. Deployment Configuration
15. Demo Credentials

List of Figures

Figure 3.1 - Master architecture overview of frontend, backend, security, and storage layers.
Figure 3.2 - End-to-end system architecture flow across application tiers.
Figure 3.3 - Detailed component-level architecture for integration and execution paths.
Figure 5.1 - Authentication and JWT token lifecycle from login to protected requests.
Figure 8.1 - File upload, persistence, and response flow for authenticated users.
Figure 9.1 - Frontend-to-backend request pipeline with interceptor and security layers.
Figure 13.1 - Login screen used for credential-based access.
Figure 13.2 - Registration screen for onboarding new users.
Figure 13.3 - Root application view after route resolution.
Figure 13.4 - Structural UI diagram of login page components and flow.
Figure 13.5 - Structural UI diagram of registration page components and flow.
Figure 13.6 - Structural UI diagram of dashboard composition and actions.
Figure 14.1 - Backend setup and runtime verification sequence for local environment.

1. Project Overview

The Secure File Management System is a full-stack CRM-oriented application built to provide secure file upload, retrieval, download, and management capabilities using stateless JWT authentication and role-based authorization.

Key Features

Feature | Description
Authentication | JWT-based login and registration with token validation filter chain
Role Isolation | ADMIN and CUSTOMER roles with separate route and API access boundaries
File Management | Upload, list, download, and delete operations tied to authenticated user context
Admin Operations | Admin dashboard endpoints for global user/file/statistics visibility
Session Handling | Auto-restore from localStorage and auto-logout on unauthorized responses

Business Rules

- Only authenticated users can access protected APIs.

- Admin APIs are accessible only by ADMIN role.
- User file listing is scoped to authenticated principal identity.
- 401 and 403 responses clear session and force re-login.

2. Technology Stack

Backend

Technology | Version | Purpose
Java | 17 | Programming language
Spring Boot | 3.2.x | Application framework
Spring Security | 6.x | Authentication and authorization
Spring Data JPA | 3.x | ORM and repository abstraction
JJWT | 0.12.x | JWT generation and validation
MySQL | 8.x | Relational data storage
Maven | 3.x | Build and dependency management

Frontend

Technology | Version | Purpose
React | 18.x | UI library
TypeScript | 5.x | Type-safe frontend code
Vite | 5.x | Dev server and bundler
React Router | 6.x | Client-side routing
Axios | 1.x | HTTP client with interceptors
Tailwind + shadcn/ui | current | Component styling and UI primitives

Runtime Ports

- Frontend: http://localhost:3000
- Backend API: http://localhost:8080/api

3. System Architecture

The platform follows a standard three-layer architecture:

Presentation Layer

- React SPA with role-aware navigation and protected routes.
- AuthContext handles token/user state and session restoration.

Application Layer

- Spring REST controllers for auth/file/admin modules.
- Service layer for business rules and orchestration.
- Security chain with JWT filter before username/password authentication filter.

Data Layer

- MySQL stores users and file metadata.
- Server filesystem stores file binaries in uploads directory.

Architecture Highlights

- Stateless API design with Bearer token model.
- Centralized axios interceptors for auth header injection and global 401/403 handling.
- Clear mapping between DTOs and entities.

Related Diagrams

- images/master-architecture.png
- images/system-architecture-flow.png
- images/architecture-detailed-view.png

Figure 3.1: Master architecture overview of frontend, backend, security, and storage layers.

Figure 3.2: End-to-end system architecture flow across application tiers.

![Detailed Architecture View](architecture-detailed-view.png)
Figure 3.3: Detailed component-level architecture for integration and execution paths.

4. Entity Relationship Diagram

Core Entities

User
- id (PK)
- email (unique)
- password
- firstName
- lastName
- role (ADMIN, CUSTOMER)
- avatar
- createdAt
- updatedAt

FileEntity
- id (PK)
- name
- type
- size
- filePath
- uploadedAt
- user (FK)

Relationships
Relationship | Type | Description
User -> FileEntity | One-to-Many | One user owns multiple files
FileEntity -> User | Many-to-One | Each file belongs to exactly one user

Constraints
- User.email is unique.
- FileEntity.user must reference a valid user id.
- Ownership checks are derived from authenticated principal.

5. Authentication & JWT Flow

Authentication Sequence
1. User submits credentials to /auth/login.
2. Backend validates credentials via authentication manager.
3. JwtUtil generates JWT containing role claim and expiry.
4. Frontend stores token and user payload in localStorage.
5. User is redirected to /admin or /dashboard based on role.

Validation Sequence
1. Axios request interceptor attaches Bearer token.
2. JwtAuthenticationFilter validates signature and expiration.
3. SecurityContext is set with authenticated principal.
4. Request reaches controller under role constraints.

Token Handling Notes
- Token is attached automatically from localStorage.
- Invalid/expired token triggers session reset and redirect to /login.

![Authentication and JWT Flow](authentication-flow.png)
Figure 5.1: Authentication and JWT token lifecycle from login to protected requests.

6. Role-Based Access Control

Permission Summary

Capability | ADMIN | CUSTOMER
Login | Yes | Yes
Register | Yes | Yes
View own profile | Yes | Yes
Upload file | Yes | Yes
List own files | Yes | Yes
Download own file | Yes | Yes
Delete own file | Yes | Yes
View admin users endpoint | Yes | No
View admin files endpoint | Yes | No
View dashboard stats endpoint | Yes | No

Enforcement Layers

- Frontend: ProtectedRoute role prop and route grouping in App.tsx.
- Backend: SecurityConfig requestMatchers with hasRole checks.

7. File CRUD Operations

Create (Upload)

- Endpoint: POST /files/upload
- Input: multipart/form-data with file
- Output: FileDto

Read (List/User)

- Endpoint: GET /files/user
- Output: FileDto[] scoped to current authenticated user

Read (Single)

- Endpoint: GET /files/{id}
- Output: FileDto

Delete

- Endpoint: DELETE /files/{id}
- Output: 200 OK or error payload

Download

- Endpoint: GET /files/download/{id}
- Output: binary stream with content-disposition attachment header

8. File Upload & Access Flow

Flow

1. User selects file from dashboard files page.
2. Frontend builds FormData and calls upload endpoint.
3. Backend validates token and resolves user context.
4. File is stored to server path, metadata persisted in MySQL.
5. Response returns FileDto and UI refreshes listing.

Follow-up Access

- User can list uploaded files via /files/user.
- User can download or delete permitted files.

Related Diagram

- images/file-upload-flow.png

Figure 8.1: File upload, persistence, and response flow for authenticated users.

9. Frontend-Backend Communication

Request Pipeline

1. UI action invokes service method.
2. Service uses shared axios instance.
3. Request interceptor injects Authorization header.
4. Request is sent to http://localhost:8080/api.
5. Spring Security validates token and role.
6. Controller -> Service -> Repository flow executes.
7. JSON (or binary for download) response is returned.

Error Handling

- 401/403: frontend clears auth_token and auth_user.
- Browser redirects to /login.

Integration Diagram

- images/frontend-backend-integration.png

Figure 9.1: Frontend-to-backend request pipeline with interceptor and security layers.

10. API Endpoints Reference

Authentication

Method	Endpoint	Description
POST	/auth/login	Authenticate and return JWT + user
POST	/auth/register	Register user and return auth payload
GET	/auth/profile	Return current user profile

Files

Method	Endpoint	Description
POST	/files/upload	Upload file for authenticated user
GET	/files/user	List files of authenticated user
GET	/files/{id}	Fetch metadata for specific file
DELETE	/files/{id}	Delete file by id
GET	/files/download/{id}	Download file binary

Admin

Method	Endpoint	Description
GET	/admin/users	List users (admin only)
GET	/admin/files	List files (admin only)
GET	/admin/dashboard-stats	Dashboard metrics (admin only)

11. Database Schema

Users Table

- id BIGINT PK
- email VARCHAR UNIQUE
- password VARCHAR
- first_name VARCHAR
- last_name VARCHAR
- role VARCHAR
- avatar VARCHAR
- created_at TIMESTAMP
- updated_at TIMESTAMP

Files Table

- id BIGINT PK

- name VARCHAR
- type VARCHAR
- size BIGINT
- file_path VARCHAR
- uploaded_at TIMESTAMP
- user_id BIGINT FK -> users.id

Storage Design

- Database stores metadata and ownership mappings.
- Filesystem stores binary file contents.

12. Security Architecture

Security Components

- SecurityConfig
- JwtAuthenticationFilter
- JwtUtil
- PasswordEncoder (BCrypt)

Security Controls

- SessionCreationPolicy.STATELESS
- Endpoint-level role restrictions
- JWT signature validation before route execution
- CORS configuration for development origins

Hardening Recommendations

- Move jwtSecret and DB credentials to environment variables.
- Add refresh token mechanism.
- Add file content validation and malware scanning.
- Add audit logs for high-risk operations.

13. Frontend Pages & Components

Primary Routes

- /login
- /register
- /admin
- /admin/users
- /admin/files
- /dashboard
- /dashboard/files
- /dashboard/profile

Key Components

- AppLayout
- AppSidebar
- ProtectedRoute
- AuthContext provider

UI Assets

- images/ui-login.png
- images/ui-register.png
- images/ui-root.png
- images/ui-login-screen.png
- images/ui-register-screen.png
- images/ui-dashboard-screen.png

Figure 13.1: Login screen used for credential-based access.

Figure 13.2: Registration screen for onboarding new users.

Figure 13.3: Root application view after route resolution.

Figure 13.4: Structural UI diagram of login page components and flow.

Figure 13.5: Structural UI diagram of registration page components and flow.

Figure 13.6: Structural UI diagram of dashboard composition and actions.

14. Deployment Configuration

Local Setup

Frontend

- cd frontend
- pnpm install (or npm install)
- pnpm dev (or npm run dev)

Backend

- cd backend
- mvn spring-boot:run

Configuration Highlights

- server.port=8080
- server.servlet.context-path=/api
- frontend Vite dev port=3000
- frontend API base URL=http://localhost:8080/api

Environment Recommendations

- Use reverse proxy + HTTPS in production.
- Restrict CORS to trusted domains.
- Use managed DB backups and log retention.

Figure 14.1: Backend setup and runtime verification sequence for local environment.

15. Demo Credentials

Source

- Refer to DEMO_CREDENTIALS.txt in project root.

Validation Checklist

- Admin login works.
- Customer login works.
- Upload/list/download/delete flow works.
- Unauthorized requests redirect to login.
- Admin routes reject non-admin role users.

Appendix: Important Code References

A) frontend/src/services/api.ts

```
export const API_BASE_URL = "http://localhost:8080/api";

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("auth_token");
  if (token) config.headers.Authorization = ‘Bearer ${token}‘;
  return config;
});
```

B) frontend/src/contexts/AuthContext.tsx

```
const login = useCallback(async (email: string, password: string) => {
  const response = await authService.login({ email, password });
  setToken(response.token);
  setUser(response.user);
  localStorage.setItem("auth_token", response.token);
  localStorage.setItem("auth_user", JSON.stringify(response.user));
}, []);
```

C) backend/src/main/java/com/example/filemanagement/security/JwtUtil.java

```
public String generateToken(UserDetails userDetails, String role) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("role", role);
    return createToken(claims, userDetails.getUsername());
}
```

D) backend/src/main/java/com/example/filemanagement/config/SecurityConfig.java

```
.authorizeHttpRequests(authz -> authz
    .requestMatchers(HttpMethod.POST, "/auth/login", "/auth/register").permitAll()
    .requestMatchers(HttpMethod.GET, "/admin/**").hasRole("ADMIN")
    .anyRequest().authenticated()
)
```

E) backend/src/main/java/com/example/filemanagement/controller/FileController.java

```
@PostMapping("/upload")
@GetMapping("/user")
@DeleteMapping("/{fileId}")
@GetMapping("/download/{fileId}")
```

End of Document.